Residency Day 2: Project 3: Secure Software Design

Group 2

Bilal Khalid

Nischal Pokharel

Samia Zaman

Syed Kashif Ali

Steven Sisjayawan

Amer Mohammed

University of the Cumberlands

MSCS-535 Secure Software Development

2026 Spring

Prof. Brandon Bass

March 28, 2026

Table of Content

Pg No.

Step 1: Accept and validate the request.	**3**
Step 2: Normalize input and start secure processing.	**4**
Step 3: Store payment details safely in the database.	**4**
Step 4: Render the response safely in the browser.	**5**
Step 5: Handle errors safely	**5**
Step 6: Supporting validation function	**6**
Step 7: How this prevents SQL injection	**7**
Step 8: How this prevents XSS	**7**
Step 9: Extra good practices for a payment system	**7**
Reference's	**8**

Below is pseudocode for an online payment flow that defends against SQL injection and cross-site scripting (XSS).

Step 1: Accept and validate the request

```
function handlePaymentRequest(request, response):
    requireHTTPS(request)
    requireCSRFToken(request)
    requireAuthenticatedUser(request)
```

```
    input = {
        cardToken: request.body.cardToken,
        amount: request.body.amount,
        currency: request.body.currency,
        billingName: request.body.billingName,
        email: request.body.email,
        note: request.body.note
    }
```

```
    if not validatePaymentInput(input):
        return response.sendError(400, "Invalid payment data")
```

Explanation:

- This block receives data from the user and checks whether it is valid before using it.
- validatePaymentInput(input) helps reduce SQL injection risk by rejecting unexpected or badly formatted values early.
- Input validation does not replace secure database queries, but it adds an extra defense layer.

Step 2: Normalize input and start secure processing

```
    safeInput = normalizeInput(input)
    beginTransaction()
```

```
    try:
        paymentResult = paymentGateway.charge(
            token = safeInput.cardToken,
            amount = safeInput.amount,
            currency = safeInput.currency
        )
```

Explanation:

- normalizeInput(input) cleans and standardizes input data, such as trimming spaces and formatting text safely.
- The payment gateway uses a token instead of raw card data, which improves application payment security.
- Starting a transaction ensures the payment record is only saved if all steps succeed.

Step 3: Store payment details safely in the database

```
        db.execute(
            "INSERT INTO payments (user_id, amount, currency, billing_name,
email, note, gateway_ref)
             VALUES (?, ?, ?, ?, ?, ?, ?)",
            [
                request.user.id,
                safeInput.amount,
                safeInput.currency,
                safeInput.billingName,
                safeInput.email,
                safeInput.note,
                paymentResult.reference
            ]
        )
```

Explanation:

- This is the main protection against SQL injection.
- The query uses placeholders ? instead of directly joining user input into the SQL string.
- User values are passed separately as parameters, so malicious input like SQL commands is treated as data, not executable SQL.

Step 4: Render the response safely in the browser

```
        commitTransaction()
```

```
        response.setHeader("Content-Security-Policy", "default-src 'self';
script-src 'self'")
        response.render("payment_success", {
            billingName: encodeForHTML(safeInput.billingName),
            email: encodeForHTML(safeInput.email),
            message: encodeForHTML("Payment completed successfully")
        })
```

Explanation:

- This block protects against cross-site scripting (XSS).
- encodeForHTML(...) converts dangerous characters like < and > into safe text before showing user data on the page.
- The Content-Security-Policy header adds another security layer by limiting which scripts the browser can run, but CSP should be used as defense in depth rather than the only XSS control.

Step 5: Handle errors safely

```
    catch error:
        rollbackTransaction()
        logSecurityEvent("Payment processing failure", error)
```

```
        return response.sendError(500, "Payment failed")
```

Explanation:

- If something goes wrong, the transaction is rolled back so incomplete payment data is not stored.
- Detailed errors are logged internally, while the user only sees a generic message.
- This avoids exposing sensitive system details that attackers could use.

Step 6: Supporting validation function

```
function validatePaymentInput(input):
    if input.amount <= 0:
        return false
```

```
    if input.currency not in ["USD", "EUR", "GBP"]:
        return false
    if not matchesRegex(input.email, EMAIL_PATTERN):
        return false
    if length(input.billingName) > 100:
        return false
    if length(input.note) > 500:
        return false
    return true
```

Explanation:

- This function checks that input values are the correct type, format, and size.
- For example, email must match a valid pattern, and text fields cannot exceed safe limits.
- This supports security by reducing unexpected data input before it reaches the application database or the browser.

Step 7: How this prevents SQL injection

Use parameterized queries with placeholders like ? instead of building SQL with string concatenation.

```
Never do this:
  - sql = "SELECT * FROM users WHERE email = '" + email + "'"
Always do this:
  - db.execute("SELECT * FROM users WHERE email = ?", [email])
```

Validate type and format early so unexpected payloads are rejected before reaching the database. Run the app with a database account that has least privilege so even a missed flaw has limited impact.

Step 8: How this prevents XSS

- Encode all untrusted data with encodeForHTML() before rendering it into pages.
- Do not insert raw user input into HTML, JavaScript, or attributes.
- Add a Content-Security-Policy header to reduce the chance that injected scripts can execute.
- If rich text is ever allowed, use a strict HTML sanitizer; otherwise store and render it as plain text.

Step 9: Extra good practices for a payment system

- Use tokenized card handling through a payment gateway; do not store raw card numbers.
- Require HTTPS, CSRF protection, authentication, rate limiting, and audit logging.
- Return generic errors to users and keep detailed errors only in server logs.
- Scan and test regularly with secure code review, SAST/DAST, and dependency updates.

References

Richardson, T., & Thies, C. N. (2012). *Secure software design* (1st ed.). Jones & Bartlett Learning.